YOLOv11 기반 고정밀 응급차량 식별 알고리즘

SAFECAR

http://mbc-sw.iptime.org:3204

MBC 아카데미 | 4조

목차

01 프로젝트 소개

02 팀원 소개

03 시스템 아키텍처

04 데이터베이스 설계

05 프론트엔드

06 알고리즘 제작 과정

07 프로젝트 시연 안내

08 Q & A 및 마무리

추진 배경 및 수요 조사

응급차량 이동 지연

도심 교통 체증으로 응급차 속도가 감소해 **골든타임 확보 실패 사례**가 빈번하게 발생하고 있습니다.

사회적 비용 증가

구조 지연은 연간 수천억 원 규모의 막대한 사회적 손실과 직결되며, 인명 피해 규모를 키우고 있습니다.

프로젝트 소개 및 목표

GOLDEN TIME

골든타임을 확보하라

기존 블랙박스 및 CCTV 자원을 활용한 실시간 응급차량 비전 인식 솔루션

팀원 소개

LEAD

박승호 (조장)

프로젝트 총괄

아키텍처 설계 및 AI 훈련 담당

DB & CO-OP

이재정 (부조장)

DB 설계

Notion, GitHub, Jira 협업 관리, AI 훈련 담당

BACKEND

유진설

분석 게시판 CRUD 구현

FRONTEND

김사브리나

Flask 기반 웹 디자인

BACKEND

조정화

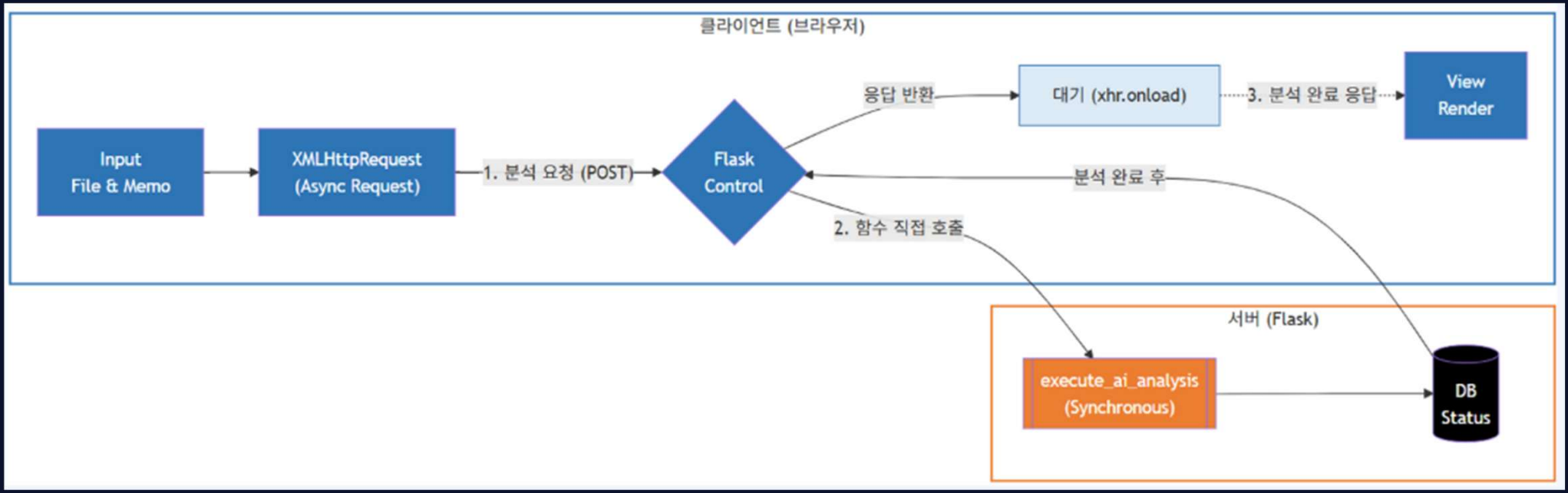
회원관리 (CRUD) 구현

BACKEND

김수빈

회원관리 (CRUD) 구현

시스템 아키텍처


클라이언트 (브라우저)
Input
File & Memo
XMLHttpRequest
(Async Request)
1. 분석 요청 (POST)
Flask
Control
응답 반환
대기 (xhr.onload)
3. 분석 완료 응답
View
Render
분석 완료 후
2. 함수 직접 호출
서버 (Flask)
execute_ai_analysis
(Synchronous)
DB
Status

DATABASE 주요기능

members

role	ENUM: 권한 분리
email	VARCHAR: 무결성 검증
active	TINYINT: 활성 상태

boards

member_id	INT : 외래키 보호
readcount	INT : 조회수 랜더링
regdate DATETIME	: 작성 시간

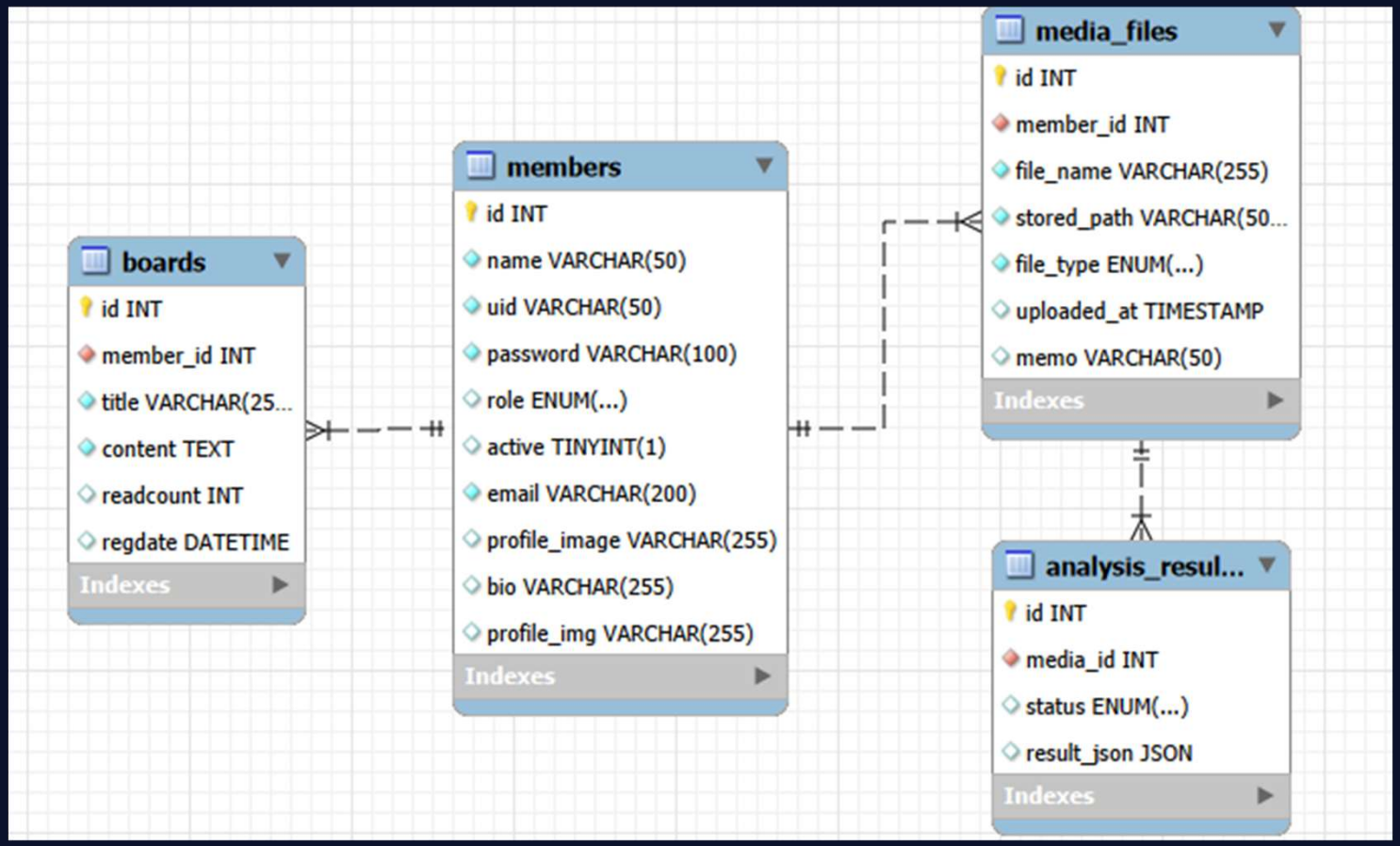
media_files

member_id	INT: 외래키 보호
file_name	: 원본 파일명
stored_path	: 서버 저장 경로
upload_at	: 업로드 일시

analysis_results

status	ENUM: 처리 상태
result_json	JSON: BBox 좌표
conf_score	FLOAT: 신뢰도
media_id	INT: 파일 연동

DATABASE (ERD)



FRONTEND(비동기 통신)

AJAX Communication

영상 업로드 시 기존 동기식 전송을 탈피하고,

XMLHttpRequest 객체를 활용하여 화면 새로고침 없는

실시간 분석 요청 및 응답 체계를 구축했습니다.

- 새로고침 없는 실시간 데이터 처리
- 동적 CSS 로딩 스피너 적용
- 사용자 인터랙션 끊김 방지

FRONTEND (렌더링 및 유효성)

Dynamic Rendering

Session 조건부 뷰 제공

Flask Jinja2 연동을 통해 사용자의 세션 상태를 실시간 감지하고 로그인 여부에 따른 맞춤형 UI를 제공합니다.

Client Caching

상세 페이지 이동 시 localStorage 데이터 전달을 활용하여 렌더링 성능을 최적화했습니다.

Data Validation

용량 한계 유효성 검사

이미지 20MB / 영상 한도 등 JS 단독 블로킹을 통해 불필요한 트래픽을 서버 도달 전 차단합니다.

무결성 데이터 송출

서버 부하를 감소시키고 무결성이 보장된 데이터만 송출하여 시스템 안정성을 확보했습니다.

AI TRAINING (알고리즘 설계)

모델 선정 및 전이 학습

YOLOv11m: 실시간 빠른 추론과 높은 정확도의 최적 균형을 고려한 최신 모델 선정

Transfer Learning: COCO 사전 학습 모델 기반으로 국내 도로 데이터를 활용한 전이 학습 수행

50	16	640
EPOCHS	BATCH	IMGSZ

```python
device = 'cuda' if torch.cuda.is_available() else 'cpu'
model = YOLO("yolov11m.pt")

model.train(
    data="/content/dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16,
    name="emergency_v1",
    exist_ok=True,
    close_mosaic=20,
    device=device,
    label_smoothing=0.1
)
```

QUANTIZATION (양자화 최적화)

서버 구동 부하 최소화

서버 실행 파이프라인에서 무거운 Pytorch 모델을 감지할 시 **반정밀도(FP16)를 지원하는 ONNX 포맷**으로 추출하여 런타임 메모리 사용량을 극적으로 낮췄습니다.

```python
onnx_path = 'best (2).onnx'
pt_path = 'best (2).pt'

if not os.path.exists(onnx_path) and os.path.exists(pt_path):
    print("🚀 [최초 시작 감지] 경량화된 ONNX 모델이 없습니다. 자동 변환을 수행합니다... (수 분 소요됨)")
    try:
        temp_model = YOLO(pt_path)
        # FP16 양자화, 832 해상도 고정을 통해 서버 부하 최소화
        temp_model.export(format='onnx', half=True, imgsz=832, simplify=True)
        print("✅ [자동 변환] 완료되었습니다! 서버를 가동합니다.")
    except Exception as e:
        print(f"⚠️ 모델 변환 중 오류가 발생하여 기존 pt 모델을 사용합니다: {e}")
        onnx_path = pt_path  # 실패 시 안전하게 기존 모델로 폴백(fallback)

try:
    yolo_model = YOLO(onnx_path)  # FP16 추론이 적용된 경량 ONNX 모델 로드
except Exception as e:
    yolo_model = None
    print(f"YOLO 모델 로드 실패: {e}")
```

Demonstration (최종 시연)

실시간 AI 미디어 분석 시연

클릭 시 발표 화면 전환 및 시연 시작

경청해 주셔서 감사합니다.

Q & A